Detección de enfermedades de la papa con aprendizaje profundo: comparación de ResNet-50 y Vision Transformer con imágenes de campo

Gabriel Ángel Nizama Duque, Martín Alexis Valdiviezo Palma,
Juan José Pino Reyes, César Luis Jiménez Miranda
Maestría en Inteligencia Artificial, Unidad de Posgrado FIIS
Universidad Nacional de Ingeniería, Lima, Perú
MIA-203 Redes Neuronales y Aprendizaje Profundo — Equipo 4, Sección A

***Resumen*—Diagnosticar a tiempo las enfermedades de la papa es clave para un cultivo central en la seguridad alimentaria del Perú. Los modelos de visión suelen entrenarse con fotos de laboratorio, pero en campo hay fondos variados, manos, sombras y plásticos. Estudiamos si, con solo 2 112 imágenes de entrenamiento tomadas en campo y 7 clases desbalanceadas, conviene una red convolucional (ResNet-50) o un Vision Transformer (ViT-S/16), y cuánto aporta el preentrenamiento en ImageNet. Primero auditamos el conjunto de datos público: encontramos 55 pares de duplicados exactos (6 de ellos con etiquetas contradictorias) y casi-duplicados, y construimos una partición 70/15/15 por grupos sin fuga de datos. Comparamos 6 configuraciones, con 3 semillas cada una: ResNet-50 y ViT-S/16, preentrenados y desde cero, y dos líneas base, un Random Forest con descriptores de color y textura y una CNN pequeña. Preentrenados, ResNet-50 y ViT-S/16 alcanzan macro-F1 de 0.843 ± 0.012 y 0.853 ± 0.013 en prueba, sin diferencia significativa según la prueba de McNemar ($p \geq 0.16$). Ambos superan ampliamente al Random Forest (0.464 ± 0.021). Sin preentrenamiento, el ViT cae a 0.499 ± 0.028, por debajo de la ResNet-50 entrenada desde cero (0.642 ± 0.007). Es decir, con pocos datos el preentrenamiento importa más que la arquitectura. En los ejemplos revisados con Grad-CAM, la ResNet enfoca las regiones con lesión, mientras que la CNN pequeña a veces usa atajos del fondo. Publicamos el código, la partición, los pesos y una demo reproducible.**

***Index Terms*—aprendizaje profundo, Vision Transformer, ResNet, transferencia de aprendizaje, enfermedades de la papa, Grad-CAM, pocos datos.**

I. Introducción

La papa es uno de los cultivos más importantes del Perú y del mundo. Las enfermedades foliares, como la rancha (*Phytophthora infestans*), las virosis o el daño por insectos, reducen el rendimiento si no se detectan a tiempo. El diagnóstico visual depende de especialistas que no siempre están disponibles en campo. Por eso, un clasificador de imágenes ejecutable desde un celular podría servir como apoyo temprano.

El aprendizaje profundo ya logra exactitudes casi perfectas en conjuntos de laboratorio como PlantVillage [1], [2], con hojas aisladas sobre fondo uniforme. Sin embargo, esos modelos se degradan con fotos de campo [1]. El conjunto de Shabrina *et al.* [3] reúne 3 076 fotos de hojas de papa tomadas en campo en Java Central (Indonesia), en 7 clases. Sus líneas base reportan exactitudes de solo 0.68–0.74.

Al mismo tiempo, los Vision Transformers (ViT) [4] han desplazado a las CNN en muchos problemas. Sin embargo, se sabe que necesitan muchos datos o un preentrenamiento fuerte [4]–[6]. Esto plantea una pregunta práctica para equipos con pocos datos y poco cómputo: *¿qué arquitectura elegir y cuánto depende el resultado del preentrenamiento?*

Contribuciones.

1. **Auditoría y partición sin fuga.** Auditamos el conjunto de datos público: 110 imágenes en 55 pares de duplicados exactos, 12 de ellas con etiquetas contradictorias, y 64 imágenes en grupos de casi-duplicados. Construimos una partición 70/15/15 por grupos, reproducible con un comando.
2. **Comparación controlada.** Comparamos ResNet-50 y ViT-S/16 (preentrenados y desde cero), una CNN pequeña y un Random Forest, con 3 semillas, métricas adecuadas al desbalance (macro-F1) y prueba de significancia (McNemar).
3. **Ablación del preentrenamiento.** Cuantificamos que aporta +0.355 de macro-F1 al ViT y +0.201 a la ResNet.
4. **Interpretabilidad y demo.** Analizamos los errores con Grad-CAM, incluyendo una medida de concentración de los mapas, y publicamos el modelo final en una app web que lo ejecuta en el celular, además de una demo reproducible en Colab.

El resto del artículo se organiza así: la Sección II revisa trabajos relacionados; la Sección III describe la metodología; la Sección IV presenta los resultados, que se discuten

en la Sección V; la Sección VI concluye.

II. Trabajos relacionados

CNN y transferencia de aprendizaje. Las redes residuales [7] permitieron entrenar CNN profundas mediante conexiones de identidad. Su preentrenamiento en ImageNet [8] es el punto de partida estándar con pocos datos. En plantas, Mohanty *et al.* [1] y Ferentinos [2] alcanzaron más de 99 % de exactitud en PlantVillage, pero advierten que el desempeño cae con imágenes de campo. Nuestro trabajo usa precisamente fotos de campo.

Transformers en visión. El Transformer [9] reemplaza la convolución por autoatención. El ViT [4] lo aplica a parches de 16×16 píxeles y supera a las CNN solo cuando se preentrena en conjuntos muy grandes. DeiT [5] y Steiner *et al.* [6] mostraron que la regularización y el preentrenamiento compensan en parte la falta de sesgo inductivo. Raghu *et al.* [10] encontraron que el ViT integra información global desde las primeras capas, a diferencia de la CNN. Nosotros medimos este contraste en un problema real con solo $\sim$2 100 imágenes.

Papa en condiciones no controladas. Shabrina *et al.* [3] publicaron el conjunto que usamos y reportaron líneas base con una sola partición 90/10: ResNet-50 obtiene 0.682 de exactitud y EfficientNetV2-B3, 0.736. Sangar y Rajasekar [11] combinan EfficientNet-LITE con un SVM en entornos diversos. Ninguno de los dos trabajos controla duplicados entre particiones, reporta varias semillas ni compara con Transformers.

Interpretabilidad y atajos. Grad-CAM [12] localiza las regiones que más influyen en una clase. Su aplicación a ViT es menos fiel que en CNN [13], lo que consideramos al interpretar. Geirhos *et al.* [14] describen el *aprendizaje por atajos*: decidir por pistas espurias del fondo. Lo observamos en nuestra CNN pequeña.

III. Metodología

La Fig. 1 resume el flujo del estudio: auditoría y limpieza de los datos, partición sin fuga, entrenamiento de los modelos y evaluación única en prueba.

III-A. Datos

Usamos el *Potato Leaf Disease Dataset in Uncontrolled Environment* [3], [15] (Mendeley Data, licencia CC BY 4.0). Contiene 3 076 fotos JPG de 1500×1500 píxeles tomadas en campo. Las 7 clases son bacteria, hongos (*fungi*), sana, nematodos, plaga de insectos (*pest*), *Phytophthora* y virus. El desbalance es marcado: de 67 imágenes (nematodos) a 740 (hongos) tras la limpieza. La Fig. 2 muestra un ejemplo por clase y la distribución por partición. Las fotos incluyen manos, plásticos de cobertura y varias hojas por imagen.

Auditoría. Calculamos el hash sha256 de cada archivo y encontramos 110 imágenes que forman 55 pares de duplicados exactos. En 6 pares, la misma foto aparece en dos clases distintas. Eliminamos esas 12 imágenes, por tener etiqueta contradictoria, y 49 copias redundantes, de modo que quedan 3 015. Además, calculamos un *hash* perceptual dHash de 64 bits sobre la versión de 256×256. Unimos con *union-find* los pares con distancia de Hamming ≤ 4, lo que da 64 imágenes en grupos de casi-duplicados.

Partición. Hacemos una partición estratificada por grupo 70/15/15 (semilla 42): 2 112 imágenes de entrenamiento, 450 de validación y 453 de prueba. Cada grupo de casi-duplicados cae completo en una sola partición, para que el modelo no se evalúe con variantes de fotos que ya vio. La validación se usa para elegir la mejor época y el modelo final; la prueba se evalúa una única vez por corrida.

III-B. Modelos

La Fig. 3 resume las tres redes.

Línea base clásica. Un Random Forest [16] con 1 834 descriptores por imagen:

- Histograma HSV de 48 dimensiones.
- Medias y desviaciones RGB/HSV (12).
- HOG [17] sobre 128×128 (1 764).
- LBP uniforme [18] (10).

Usa pesos de clase balanceados y una búsqueda en malla (número de árboles, `max_features` y `min_samples_leaf`) elegida por macro-F1 de validación.

CNN pequeña (1.2 M parámetros), entrenada desde cero. Tiene 4 bloques [Conv3×3–BN–ReLU]×2 + MaxPool, con anchos 32–256, seguidos de *pooling* global, *dropout* 0.3 y una capa lineal.

ResNet-50 [7] (23.5 M). Cada bloque aprende un residuo, $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$; la convolución impone localidad e invariancia a la traslación.

ViT-S/16 [4] (21.7 M). Divide la imagen de 224×224 en $N = 196$ parches de 16×16 y los proyecta a $d = 384$ dimensiones. Agrega un token `[CLS]` y embeddings de posición, y aplica 12 bloques de autoatención multi-cabeza (6 cabezas) con

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V, \qquad (1)$$

donde cada parche puede atender a todos los demás desde la primera capa.

Pesos preentrenados. Ambos modelos grandes provienen de `timm` [19]:

- ResNet-50: `resnet50.a1_in1k`, preentrenada en ImageNet-1k.
- ViT-S/16: `vit_small_patch16_224.augreg_in21k_ft_in1k`, preentrenado en ImageNet-21k y ajustado en ImageNet-1k.

Reemplazamos la cabeza por una capa de 7 salidas y ajustamos *todos* los parámetros. En la ablación "desde cero" usamos la misma arquitectura con inicialización aleatoria.

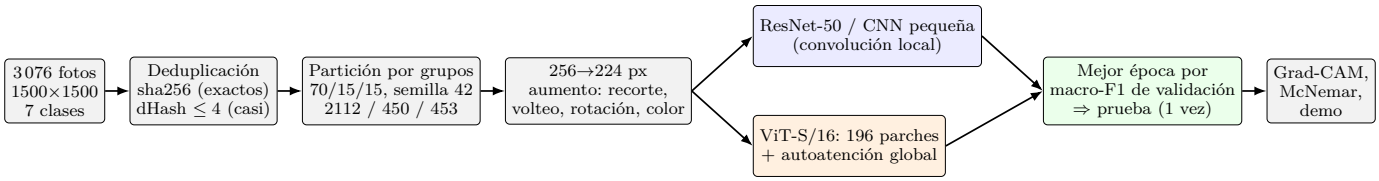


Figura 1. Flujo del estudio. La partición se hace por grupos de casi-duplicados para evitar fuga de datos; el conjunto de prueba solo se usa una vez, al final de cada corrida.

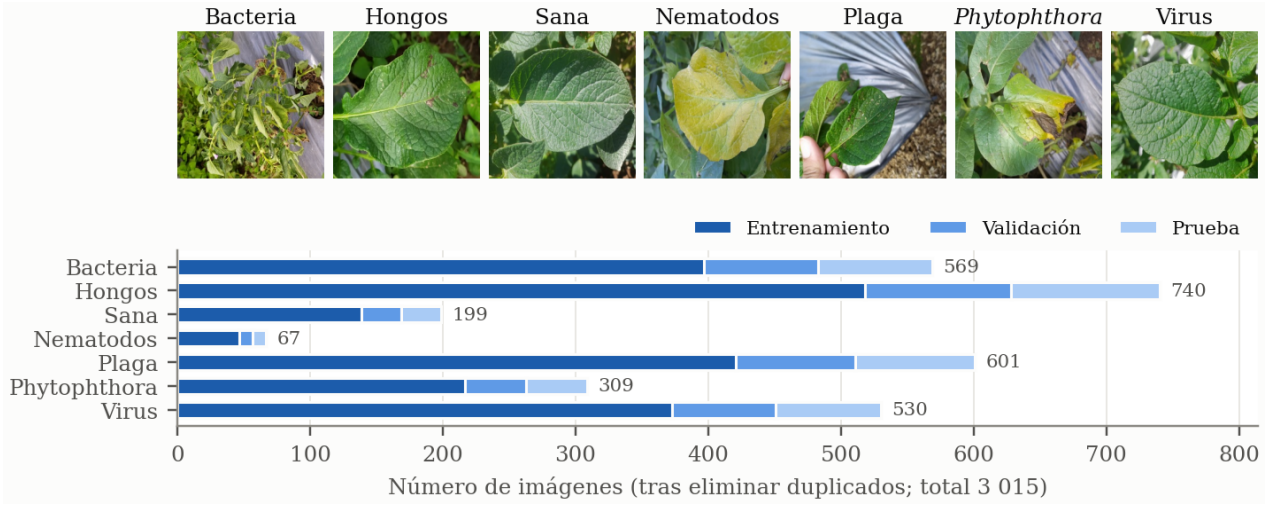


Figura 2. Conjunto de datos. Arriba: una imagen de prueba por clase, tomada en campo con fondos no controlados (plástico, manos, otras plantas). Abajo: número de imágenes por clase y partición tras eliminar duplicados; la clase mayor (hongos) tiene 11 veces más imágenes que la menor (nematodos).

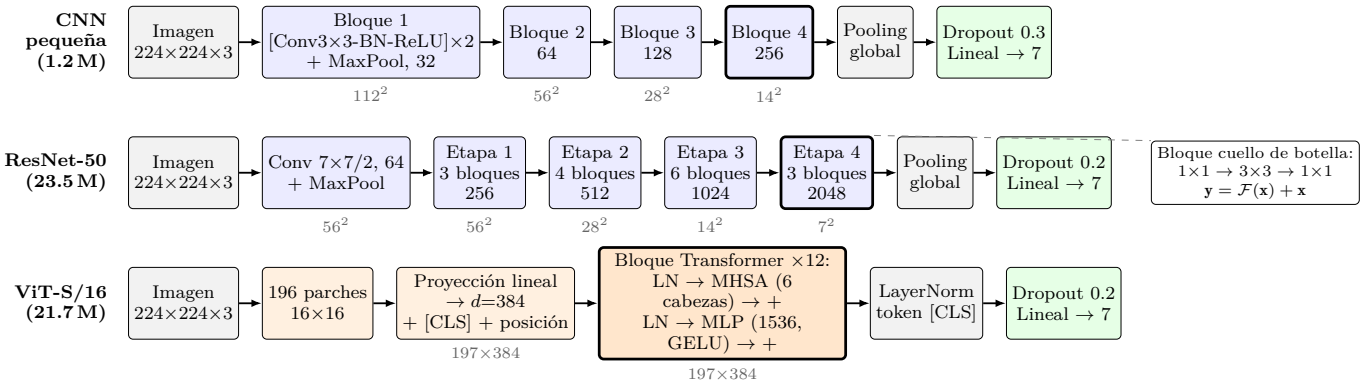


Figura 3. Arquitecturas comparadas. Las cifras bajo cada caja son el tamaño espacial del mapa ($h\times w$) o de la secuencia de *tokens*. El borde grueso marca la capa donde se calcula Grad-CAM. Los tres modelos terminan en una capa lineal de 7 salidas seguida de *softmax*.

III-C. Función de pérdida y entrenamiento

Para compensar el desbalance usamos entropía cruzada ponderada:

$$\mathcal{L} = -\frac{\sum_{i=1}^{B} w_{y_i} \log p_{i,y_i}}{\sum_{i=1}^{B} w_{y_i}}, \qquad p_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=1}^{K} e^{z_{i,j}}}, \tag{2}$$

con pesos $w_k = \frac{1/n_k}{\frac{1}{K}\sum_j 1/n_j}$, donde n_k es el número de imágenes de la clase k en entrenamiento y $K = 7$. La pérdida es la media ponderada del minibatch de tamaño B, que es lo que calcula `CrossEntropyLoss` de PyTorch cuando recibe pesos. Así, la clase nematodos (47 imágenes) pesa 3.5 y la clase hongos, 0.32.

Optimizador y calendario. Usamos AdamW [20] (Tabla I), con tasas de aprendizaje tomadas de configuraciones estándar de la literatura [4], [6] y verificadas en validación. La tasa de aprendizaje sube linealmente durante 2 épocas y luego decae con coseno hasta cero:

$$\eta_t = \begin{cases} \eta_{\max}\,\frac{t+1}{T_w}, & t < T_w \\ \frac{\eta_{\max}}{2}\left(1 + \cos\frac{\pi(t-T_w)}{T-T_w}\right), & t \geq T_w \end{cases} \tag{3}$$

Tabla I
HIPERPARÁMETROS (ADAMW, LOTE 32, 40 ÉPOCAS, 224×224)

Modelo	$\eta_{\text{máx}}$ (pre / cero)	*Weight decay*	Parámetros
CNN pequeña	– / 10^{-3}	0.01	1.2 M
ResNet-50	3×10^{-4} / 10^{-3}	0.05	23.5 M
ViT-S/16	10^{-4} / 5×10^{-4}	0.05	21.7 M

con t en pasos de minibatch.

Configuración común. Tamaño de lote 32, 40 épocas sin parada temprana, recorte de gradiente a norma 1 y *dropout* en la cabeza: 0.2 en ResNet y ViT, y 0.3 en la CNN pequeña. La precisión mixta fp16 se usa en ResNet y ViT; la CNN pequeña entrena en fp32 porque en fp16 colapsaba en nuestra GPU local.

Aumento de datos. Recorte aleatorio redimensionado (escala 0.6–1), volteos horizontal y vertical, rotación de ±20° y variación de color: brillo, contraste y saturación (±20 %) y tono (±0.02). Cada configuración se entrena con las semillas 0, 1 y 2.

III-D. Métricas y prueba estadística

Como las clases están desbalanceadas, la métrica principal es el **macro-F1**, $\frac{1}{K}\sum_k \mathrm{F1}_k$, que da el mismo peso a cada clase. También reportamos la exactitud balanceada y la exactitud, como media ± desviación estándar sobre semillas.

Para comparar ResNet-50 y ViT-S/16 con la misma semilla aplicamos la prueba exacta de McNemar [21]. Sean b y c las imágenes de prueba que solo acierta uno u otro modelo; bajo H_0, $b \sim \text{Binomial}(b+c, \frac{1}{2})$.

III-E. Interpretabilidad

Aplicamos Grad-CAM [12]:

$$\alpha_k^c = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c = \mathrm{ReLU}\Big(\sum_k \alpha_k^c A^k\Big). \tag{4}$$

Las capas objetivo son:

- ResNet-50: el último bloque de `layer4` (mapa de 7×7).
- CNN pequeña: la última convolución.
- ViT: la normalización previa al último bloque; descartamos el token `[CLS]` y reordenamos los 196 tokens en una rejilla de 14×14.

Para cuantificar la forma de los mapas medimos dos cosas:

- La *masa top-10 %*: fracción de la activación que cae en el 10 % de píxeles más activos. Un mapa uniforme da 0.10.
- La *masa central*: fracción en el cuadrado central, que ocupa el 25 % del área. Un mapa uniforme da 0.25.

IV. RESULTADOS

IV-A. Comparación con las líneas base

La Tabla II resume las 18 corridas propias (6 configuraciones × 3 semillas).

Tabla II
RESULTADOS EN PRUEBA (453 IMÁGENES), MEDIA ± DE SOBRE 3 SEMILLAS. **Negrita**: MEJOR.

Modelo	Macro-F1	Exact. bal.	Exactitud
Random Forest (color+HOG+LBP)	0.464 ± 0.021	0.452 ± 0.014	0.558 ± 0.010
CNN pequeña (cero)	0.614 ± 0.011	0.661 ± 0.004	0.630 ± 0.009
ResNet-50 (cero)	0.642 ± 0.007	0.687 ± 0.013	0.659 ± 0.003
ViT-S/16 (cero)	0.499 ± 0.028	0.550 ± 0.019	0.497 ± 0.029
ResNet-50 (preentrenada)	0.843 ± 0.012	0.845 ± 0.011	0.855 ± 0.005
ViT-S/16 (preentrenado)	$\mathbf{0.853\pm0.013}$	$\mathbf{0.849\pm0.008}$	$\mathbf{0.869\pm0.016}$
Publicado [3]: partición 90/10 sin deduplicar, 1 corrida			
ResNet-50	–	–	0.682
EfficientNetV2-B3	–	–	0.736

Tabla III
MCNEMAR EXACTO, RESNET-50 VS VIT-S/16 PREENTRENADOS (PRUEBA)

Semilla	Solo ResNet acierta (b)	Solo ViT acierta (c)	p
0	20	27	0.38
1	24	25	1.00
2	20	31	0.16

- **Frente a la línea base clásica:** los dos modelos preentrenados superan al Random Forest en más de 0.37 de macro-F1, y a la CNN pequeña en más de 0.22.
- **Frente a lo publicado:** nuestra ResNet-50 preentrenada obtiene 0.855 de exactitud, contra 0.682 en [3]. Hay que leer esta comparación con cautela, porque la partición y el protocolo difieren.
- **Variabilidad entre semillas:** es baja (≤ 0.013 de DE en macro-F1) para los modelos preentrenados.

IV-B. CNN frente a ViT

Con preentrenamiento, el ViT-S/16 supera a la ResNet-50 por solo 0.010 de macro-F1, una diferencia menor que la desviación entre semillas. La prueba de McNemar por semilla (Tabla III) no rechaza H_0 en ningún caso ($p = 0.38$, 1.00 y 0.16).

Por clase (Fig. 4), los dos modelos se reparten las victorias: el ViT es mejor en *Phytophthora* (0.87 frente a 0.80) y hongos (0.85 frente a 0.82); la ResNet, en virus (0.88 frente a 0.86) y nematodos (0.80 frente a 0.77). Concluimos que, preentrenadas, **no encontramos diferencia significativa entre ambas arquitecturas** en este problema. Esto no prueba que sean equivalentes, ya que para ello haría falta una prueba de equivalencia; pero la diferencia observada (0.010) es menor que la variación entre semillas, así que con estos datos la elección de arquitectura pesa poco.

IV-C. Ablación: efecto del preentrenamiento

La Tabla IV aísla el preentrenamiento manteniendo arquitectura, datos, partición e hiperparámetros, y ajustando solo la tasa de aprendizaje.

- **Ganancia en el ViT:** +0.355 de macro-F1.
- **Ganancia en la ResNet:** +0.201.

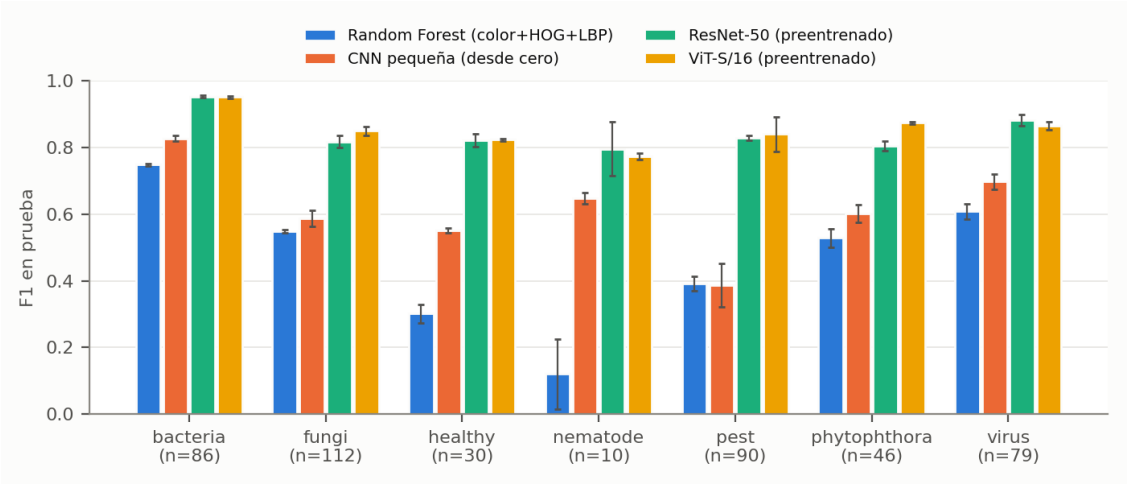


Figura 4. F1 por clase en prueba (media y DE sobre 3 semillas; n = imágenes de prueba por clase). Los modelos preentrenados dominan en todas las clases; la mayor brecha con las líneas base está en nematodos, sana y plaga. Las etiquetas se muestran con los nombres originales del dataset en inglés (*fungi*=hongos, *healthy*=sana, *nematode*=nematodos, *pest*=plaga).

Tabla IV
Ablación del preentrenamiento (macro-F1 en prueba)

Modelo	Preentrenado	Desde cero	Δ
ResNet-50	0.843 ± 0.012	0.642 ± 0.007	$+0.201$
ViT-S/16	0.853 ± 0.013	0.499 ± 0.028	$+0.355$

- **ViT desde cero:** queda incluso por debajo de la CNN pequeña, que tiene 18 veces menos parámetros, y es el modelo más inestable entre semillas (DE 0.028).
- **ResNet desde cero:** supera levemente a la CNN pequeña (0.642 frente a 0.614).

Las curvas de entrenamiento (Fig. 5) muestran la misma diferencia en la dinámica:

- **Preentrenados:** superan 0.8 de macro-F1 de validación en menos de 5 épocas y se estabilizan cerca de 0.9. La pérdida de entrenamiento llega casi a cero, pero la validación no empeora; elegir la mejor época por validación evita usar un punto sobreajustado.
- **Desde cero:** a las 40 épocas la pérdida sigue bajando y la validación sigue subiendo. Con más épocas probablemente mejorarían, así que la ablación mide la ganancia del preentrenamiento *con el mismo presupuesto de cómputo*, no su límite.

Esto coincide con [4], [5]: sin el sesgo inductivo de localidad, el Transformer necesita aprender desde los datos cómo se relacionan los parches vecinos, y 2 112 imágenes no alcanzan.

IV-D. Análisis de errores

La Fig. 6 muestra la matriz de confusión del ViT preentrenado, sumando las 3 semillas. Los errores se concentran en tres confusiones:

- **Plaga → hongos:** 14 % de las fotos de plaga. En ResNet, 11 %.
- ***Phytophthora* → hongos:** 12 %.
- **Nematodos → hongos:** 17 %, con solo 10 imágenes de prueba.

Hongos es la clase más numerosa y la de síntomas más variados (manchas necróticas, clorosis), por lo que funciona como "clase sumidero". En el Random Forest, el 34 % de las fotos de plaga se clasifica como hongos. La propia auditoría apunta a una frontera ambigua: de los 6 pares de fotos idénticas con dos etiquetas, 5 involucran a plaga y 3 son exactamente hongos/plaga (notebook de exploración). Además, el 11 % de las hojas sanas se confunde con virus, cuyos síntomas tempranos (mosaico leve) son sutiles a 224×224.

IV-E. Interpretabilidad (Grad-CAM)

La Tabla V cuantifica los mapas sobre 22 imágenes de prueba (14 aciertos, 2 por clase, y 8 errores elegidos al azar) usando la mejor corrida de cada arquitectura (CNN pequeña s2, ResNet-50 preentrenada s1 y ViT-S/16 preentrenado s2). La Fig. 7 ilustra 4 casos representativos de ese subconjunto. Cabe notar que la masa central refleja también que las hojas suelen estar centradas en la foto, y que la ResNet calcula Grad-CAM sobre una rejilla de 7×7 frente a 14×14 del ViT; al interpolarse a 224×224, un mapa más grueso tiende a concentrar mayor masa en el top-10 %.

- **ResNet-50:** produce mapas compactos (masa top-10 % = 0.50) centrados en la lesión. Por ejemplo, señala la mancha necrótica en *Phytophthora* (fila 3) y la perforación en plaga (fila 4).
- **ViT-S/16:** sus mapas son más dispersos (0.29), casi tan repartidos como un mapa uniforme en la región central (0.27 frente a 0.25). Esto es coherente con su atención global [10], aunque Grad-CAM es menos fiel

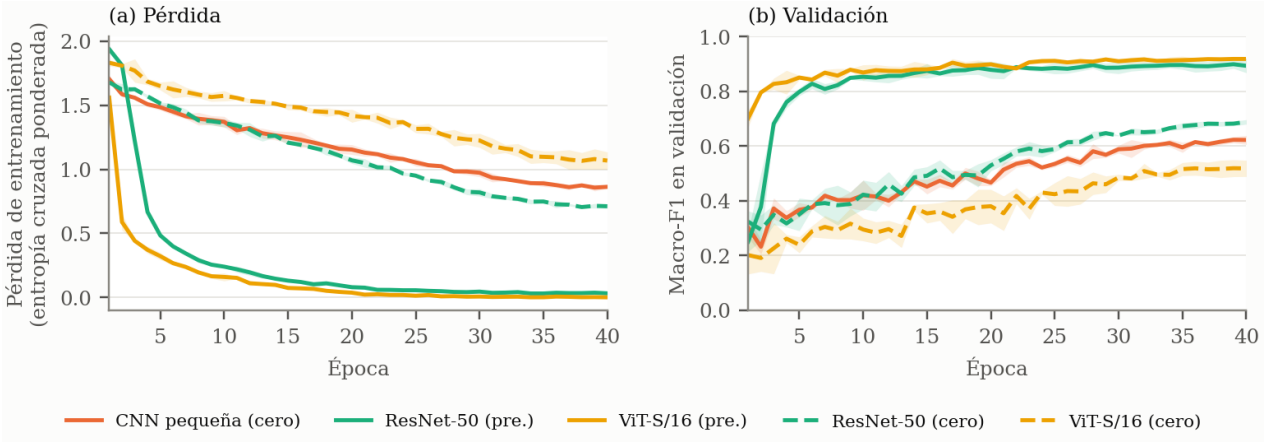


Figura 5. Curvas de entrenamiento (media y banda de ±1 DE sobre 3 semillas). (a) Pérdida de entrenamiento. (b) Macro-F1 en validación. Línea continua: preentrenado (y CNN pequeña); discontinua: desde cero. Mismo color = misma arquitectura.

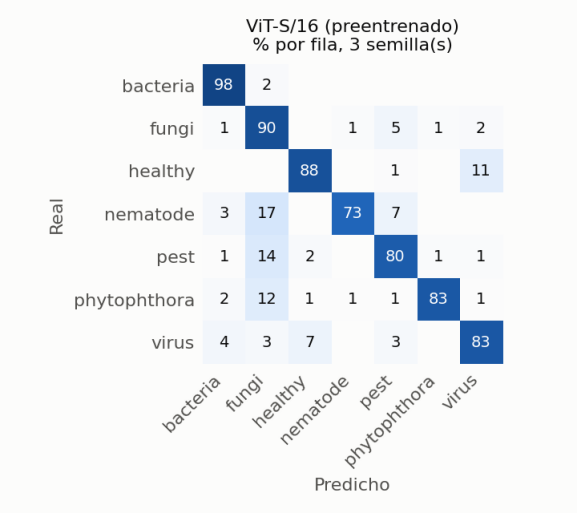


Figura 6. Matriz de confusión del ViT-S/16 preentrenado en prueba (% por fila, 3 semillas sumadas). Hongos actúa como clase sumidero de plaga, *Phytophthora* y nematodos.

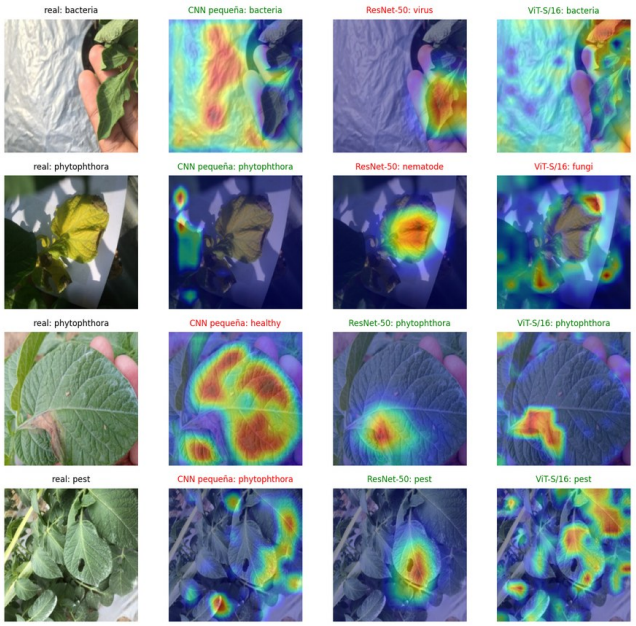


Figura 7. Grad-CAM en imágenes de prueba (verde = acierto, rojo = error). Fila 1: la CNN pequeña acierta usando el plástico del fondo (atajo). Fila 3: ResNet y ViT localizan la lesión de *Phytophthora*; la CNN pequeña no.

en Transformers [13]. Aun así, en la fila 3 se concentra en la misma lesión que la ResNet.

- **CNN pequeña:** muestra *aprendizaje por atajos* [14]. En la fila 1 acierta "bacteria" mirando el plástico de *mulch* del fondo, no la hoja. En la fila 3 predice "sana" porque atiende a las nervaduras en lugar de la lesión.

Este hallazgo advierte que, en fotos de campo, un acierto no garantiza que el modelo use la evidencia correcta.

IV-F. Costo computacional

Usamos dos equipos: Google Colab (Tesla T4, 16 GB) y una laptop con GeForce GTX 1650 Ti de 4 GB. Las 15 corridas de redes neuronales suman ≈5.9 h de GPU; el Random Forest corre en CPU (Tabla VI).

- **En la T4:** una corrida de 40 épocas tarda ≈15 min, con un pico de memoria de 1.5 GB (ViT) y 1.8 GB (ResNet).
- **En la laptop:** la misma corrida de ResNet tarda 66 min. En esa GPU, la precisión mixta fp16 produjo colapsos (la validación predecía una sola clase), por lo que la CNN pequeña se entrenó en fp32 y las semillas 1 y 2 de la ResNet preentrenada se entrenaron en la T4.
- **En inferencia:** el modelo final clasifica una imagen y calcula su Grad-CAM en ≈0.2 s en CPU.

IV-G. Modelo final y demo

Elegimos como modelo final la corrida con mayor macro-F1 de *validación*, sin mirar la prueba: ViT-S/16 preen-

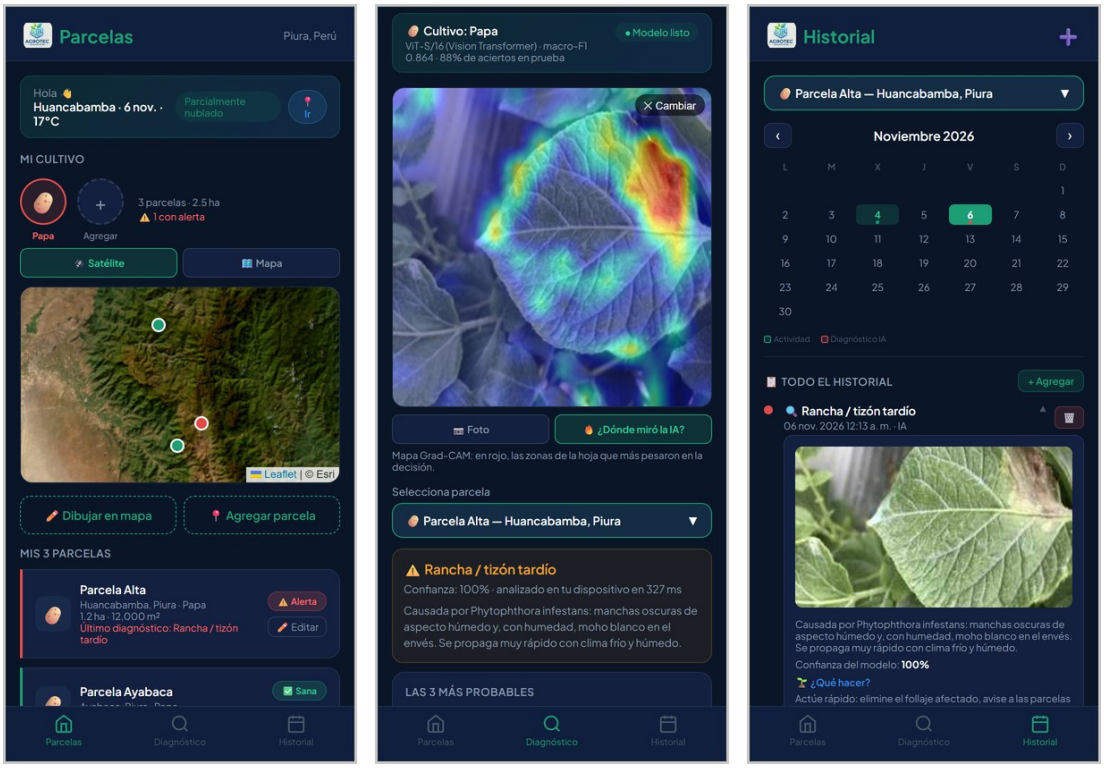


Figura 8. App web con el modelo final (capturas en un celular). Izquierda: parcelas de papa en un mapa satelital; la parcela con un diagnóstico de enfermedad pasa a "Alerta". Centro: diagnóstico de una foto de prueba de rancha, con el mapa Grad-CAM y la confianza. Derecha: historial con calendario y el diagnóstico guardado.

Tabla V
Concentración de los mapas Grad-CAM (22 imágenes de prueba)

Modelo	Masa top-10 %	Masa central
CNN pequeña	0.37 ± 0.18	0.34 ± 0.13
ResNet-50	0.50 ± 0.11	0.61 ± 0.18
ViT-S/16	0.29 ± 0.11	0.27 ± 0.11
Mapa uniforme (referencia)	0.10	0.25

Tabla VI
Costo de entrenamiento (40 épocas por corrida)

Modelo	Hardware	min/corrida	Corridas
Random Forest	CPU	1.8	3
CNN pequeña	GTX 1650 Ti	40.1	3
ResNet-50 (pre)	GTX 1650 Ti / T4	65.9 / 15.8	1 / 2
ResNet-50 (cero)	T4	15.9	3
ViT-S/16 (pre)	T4	14.4	3
ViT-S/16 (cero)	T4	15.0	3
Total GPU		≈5.9 h	

trenado con semilla 2. Obtuvo 0.929 en validación y, en prueba, 0.864 de macro-F1 y 0.883 de exactitud.

App web. Publicamos el modelo final en una aplicación web para celular, https://agrotec-papa.vercel.app (Fig. 8). Tiene tres pantallas: parcelas en un mapa satelital, diagnóstico e historial. La interfaz reutiliza el diseño de AGROTEC, un prototipo de asistencia agrícola para Piura desarrollado por uno de los autores; en esta versión, el diagnóstico lo hace el modelo de este trabajo. El modelo corre *en el navegador* con ONNX Runtime Web, sin servidor, y la foto no sale del dispositivo:

- **Exportación:** convertimos el ViT a ONNX y cuantizamos sus pesos a int8. El archivo pasa de 89 a 24 MB y en prueba obtiene 0.864 de macro-F1 y 0.883 de exactitud, igual que el modelo original.
- **Grad-CAM sin gradientes:** el navegador no calcula gradientes. Por eso escribimos en el grafo el gradiente analítico del último bloque (cabeza, LayerNorm, MLP y atención del token [CLS]); el mapa coincide con el de PyTorch (error máximo $< 10^{-5}$).
- **Salida:** la clase más probable con su confianza, las 3 más probables, el mapa Grad-CAM, una recomendación general y una advertencia cuando la confianza es menor que 0.6. Un diagnóstico tarda alrededor de 0.5 s en una laptop.

También hay una demo en Colab y otra local con Gradio, con las mismas salidas.

V. Discusión

¿Por qué gana el preentrenamiento y no la arquitectura? Con 2 112 imágenes, las representaciones apren-

didas en ImageNet (bordes, texturas, partes de objetos) son más valiosas que el sesgo inductivo de cada arquitectura. Ese sesgo solo marca diferencia cuando falta el preentrenamiento: la convolución aporta localidad "gratis" y por eso la ResNet desde cero aguanta mejor que el ViT.

Sesgos y validez. Hay que tener en cuenta varias limitaciones:

- **Generalización geográfica:** todas las fotos provienen de una región de Indonesia, con variedades, iluminación y fondos propios. No sabemos cómo se comportaría el modelo con cultivos andinos.
- **Clases pequeñas:** nematodos tiene solo 10 imágenes de prueba, así que su F1 es muy incierto.
- **Brecha validación–prueba:** el modelo final obtuvo 0.929 en validación y 0.864 en prueba. Como esta caída de 0.01 a 0.07 aparece en todos los modelos (también en el promedio de corridas no seleccionadas: ViT preentrenado 0.922 en validación frente a 0.853 en prueba; ResNet preentrenada 0.900 frente a 0.843), se debe en parte a que la partición de prueba resultó más difícil que la de validación y en parte a elegir época y modelo con la validación.
- **Comparabilidad con lo publicado:** no podemos comparar directamente con [3], porque su partición no controla duplicados.
- **Hardware mixto:** la semilla 0 de la ResNet preentrenada se entrenó en la laptop (GTX 1650 Ti) y las semillas 1 y 2 en la Tesla T4 de Colab.

Uso responsable. El sistema es una herramienta de apoyo y no reemplaza el diagnóstico de un agrónomo. Una predicción errónea puede llevar a aplicar un fungicida innecesario o a no tratar una plaga a tiempo. Por eso la demo muestra la confianza, el Grad-CAM y advierte cuando la confianza es baja. Antes de usarlo en el Perú haría falta validarlo con fotos locales.

VI. Conclusiones

En la detección de 7 enfermedades de la papa con fotos de campo y pocos datos, ResNet-50 y ViT-S/16 preentrenados no muestran diferencia significativa (macro-F1 0.84–0.85) y superan ampliamente a las líneas base. El preentrenamiento es el factor decisivo, en especial para el Transformer (+0.355). En los casos analizados, Grad-CAM muestra que la ResNet suele enfocar las lesiones mientras que una red pequeña puede apoyarse en atajos del fondo.

Como trabajo futuro proponemos:

- Recolectar fotos de papa peruana para evaluar la transferencia de dominio.
- Probar métodos de explicación específicos para Transformers [13].
- Destilar el modelo a una versión ligera para celulares.

Contribuciones

Integrante	Tareas en las que participó
Gabriel Ángel Nizama Duque	Implementación y entrenamiento de la CNN pequeña, ResNet-50 y ViT-S/16 (3 semillas), ablación del preentrenamiento, selección del modelo final y app web
Martín Alexis Valdiviezo Palma	Obtención y auditoría del conjunto de datos, eliminación de duplicados, partición 70/15/15 por grupos, apoyo en la ablación del preentrenamiento y en la app web
Juan José Pino Reyes	Línea base Random Forest, prueba de McNemar, Grad-CAM y análisis de errores
César Luis Jiménez Miranda	Apoyo en la ejecución de las corridas desde cero en Colab, repositorio y documentación, redacción del artículo y diapositivas

Disponibilidad y uso de herramientas

El código, la partición, las métricas de las 18 corridas, los pesos del modelo final y los notebooks de reproducción y demo están en https://github.com/gabo2607/papa-cnn-vs-vit. La app web está en https://agrotec-papa.vercel.app. Cada tabla y figura se regenera con los comandos del README. Se usó un asistente de IA (Claude) como apoyo en programación y redacción; el equipo revisó y validó el diseño experimental, los resultados y el texto.

Referencias

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[2] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[3] N. H. Shabrina, S. Indarti, R. Maharani, D. A. Kristiyanti, Irmawati, N. Prastomo, and T. Adilah M, "A novel dataset of potato leaf disease in uncontrolled environment," *Data in Brief*, vol. 52, p. 109955, 2024.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Int. Conf. Learning Representations (ICLR)*, 2021.

[5] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 10 347–10 357.

[6] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your ViT? data, augmentation, and regularization in vision transformers," *Transactions on Machine Learning Research*, 2022.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[10] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy, "Do vision transformers see like convolutional neural networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
[11] G. Sangar and V. Rajasekar, "Optimized classification of potato leaf disease using EfficientNet-LITE and KE-SVM in diverse environments," *Frontiers in Plant Science*, vol. 16, p. 1499909, 2025.
[12] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 618–626.
[13] H. Chefer, S. Gur, and L. Wolf, "Transformer interpretability beyond attention visualization," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 782–791.
[14] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.
[15] N. H. Shabrina, S. Indarti, R. Maharani, D. A. Kristiyanti, I. Irmawati, N. Prastomo, and T. Adillah M, "Potato leaf disease dataset in uncontrolled environment," Mendeley Data, V1, 2023, licencia CC BY 4.0.
[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[17] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.
[18] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 971–987, 2002.
[19] R. Wightman, "PyTorch image models," https://github.com/huggingface/pytorch-image-models, 2019.
[20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Int. Conf. Learning Representations (ICLR)*, 2019.
[21] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.